

ABC455 E - Unbalanced ABC Substrings 题解

题意概括

给定只由 A、B、C 组成的字符串 S ，统计有多少个非空子串满足：

- A、B、C 在这个子串中的出现次数两两不同。

解题思路

设某个子串中 A、B、C 的出现次数分别为 a 、 b 、 c 。

题目要求的是：

- $a \neq b$
- $a \neq c$
- $b \neq c$

更容易统计的是它的补集，也就是“至少有一对相等”。

第一步：容斥

所有非空子串总数为：

$$\frac{N(N+1)}{2}$$

记：

- $f(a = b)$ ：子串中 A 和 B 出现次数相等的个数；
- $f(a = c)$ ：子串中 A 和 C 出现次数相等的个数；
- $f(b = c)$ ：子串中 B 和 C 出现次数相等的个数；
- $f(a = b = c)$ ：三者都相等的个数。

那么答案为：

$$\frac{N(N+1)}{2} - f(a = b) - f(a = c) - f(b = c) + 2f(a = b = c)$$

原因是：

- 三个“相等事件”先都减掉；
- 如果三者都相等，它会被减掉 3 次，但实际上补回后应该只减掉 1 次，所以要再加回 2 次。

第二步：统计 $f(a = b)$

设前缀中 A、B 的个数分别为 $\text{prefA}[i]$ 、 $\text{prefB}[i]$ 。

对于子串 $(l + 1, \dots, r)$ ，有：

A 的个数 = $\text{prefA}[r] - \text{prefA}[l]$

B 的个数 = $\text{prefB}[r] - \text{prefB}[l]$

它们相等当且仅当：

$$\text{prefA}[r] - \text{prefB}[r] = \text{prefA}[l] - \text{prefB}[l]$$

所以只要统计每个前缀差值 $\text{prefA}[i] - \text{prefB}[i]$ 出现了多少次即可。若某个差值出现了 c 次，就能贡献：

$$\frac{c(c-1)}{2}$$

个满足条件的子串。

第三步：统计 $f(a = b = c)$

同理，三者都相等等价于：

- $\text{prefA}[r] - \text{prefB}[r] = \text{prefA}[l] - \text{prefB}[l]$
- $\text{prefA}[r] - \text{prefC}[r] = \text{prefA}[l] - \text{prefC}[l]$

因此只要把二元组：

$$(\text{prefA}[i] - \text{prefB}[i], \text{prefA}[i] - \text{prefC}[i])$$

当作一个键，统计相同键出现的次数即可。

正确性说明

容斥公式已经把“至少有一对出现次数相等”的子串个数准确扣除，所以最终得到的就是三者出现次数两两不同的子串数。

对于 $f(a = b)$ ：

- 若两个前缀的 $\text{prefA} - \text{prefB}$ 相同，那么它们之间这段子串里 A 和 B 的新增数量相同；
- 反之，若子串中 A 和 B 的数量相同，则对应两个前缀的 $\text{prefA} - \text{prefB}$ 必然相同。

因此统计相同前缀差值的前缀对数，得到的恰好就是 $f(a = b)$ 。

对于 $f(a = b = c)$ 也是同理，只是把一个差值扩展成两个差值组成的有序对。

综上，算法正确。

复杂度

每次扫描字符串只需要线性时间。

- 时间复杂度: $O(N)$
- 空间复杂度: $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

ll count_equal_pair(const string &s, char x, char y){

    int n = (int)s.size();
    int offset = n + 2;
    vector<ll> cnt(2 * offset + 1, 0);

    int diff = 0;
    cnt[offset] = 1;

    ll ans = 0;
    for(char ch : s){
        if(ch == x){
            diff++;
        }
        if(ch == y){
            diff--;
        }

        ans += cnt[diff + offset];
        cnt[diff + offset]++;
    }

    return ans;
}

ll count_equal_three(const string &s){

    int n = (int)s.size();
```

```
int offset = n + 2;
long long base = 2LL * offset + 1;

unordered_map<long long, ll> cnt;
cnt.reserve(2 * n + 5);

int ca = 0, cb = 0, cc = 0;
long long key0 = 1LL * offset * base + offset;
cnt[key0] = 1;

ll ans = 0;
for(char ch : s){
    if(ch == 'A'){
        ca++;
    } else if(ch == 'B'){
        cb++;
    } else {
        cc++;
    }

    int d1 = ca - cb;
    int d2 = ca - cc;
    long long key = 1LL * (d1 + offset) * base + (d2 + offset);

    ans += cnt[key];
    cnt[key]++;
}

return ans;
}

int main(){

    int n;
    string s;
    cin >> n >> s;

    ll total = 1LL * n * (n + 1) / 2;
    ll ab = count_equal_pair(s, 'A', 'B');
    ll ac = count_equal_pair(s, 'A', 'C');
    ll bc = count_equal_pair(s, 'B', 'C');
```

```
ll abc = count_equal_three(s);  
  
ll ans = total - ab - ac - bc + 2LL * abc;  
cout << ans;  
  
return 0;  
}
```

